Studio Nx

백엔드 281 — 패턴 라우터

2026-10-19

백엔드 281 — 패턴 라우터 (api/v1/patterns.py)

항목	내용
파일	`studionx/backend/app/api/v1/patterns.py`
작업	S1-10a 클립·연결·패턴 API: 패턴과 노트 저장의 HTTP 라우터 (PRD 02 FR-2196, FR-231)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 280 클립 · 연결 라우터, 다음 S1-10a 문서 정리

1. 하는 일

패턴 여섯 길을 한 파일에 둡니다. `app/api/v1/router.py` 에서 `/patterns` 접두사로 붙이므로, 화면은 `/api/v1/patterns` 아래의 경로로 부릅니다. 라우터는 요청 모양과 상태 코드만 정하고, 권한과 404 그리고 클립 사용 검사는 `pattern_service`(279)에 그대로 넘깁니다.

메서드와 경로	하는 일	상태 코드
`GET /patterns?project_id=`	프로젝트의 패턴 목록 (노트는 담지 않습니다)	200
`POST /patterns`	새 빈 패턴 만들기	201
`GET /patterns/{pattern_id}`	패턴 한 건과 노트 전부 (피아노롤이 열 때)	200
`PUT /patterns/{pattern_id}/notes`	노트를 보낸 목록으로 통째로 교체	200
`PATCH /patterns/{pattern_id}`	이름, 색, 길이, 스윙 수정	200
`DELETE /patterns/{pattern_id}`	패턴 삭제 (클립이 쓰면 400, 아니면 204)	204

```
POST   /patterns             PatternCreateRequest          → PatternResponse (201)
PUT    /patterns/{id}/notes  NotesSaveRequest {notes:[…]}  → PatternResponse (노트 포함)
DELETE /patterns/{id}        본문 없음                      → 204, 쓰이는 패턴이면 400
```

2. 코드

```
@router.post(
    "",
    response_model=PatternResponse,
    status_code=status.HTTP_201_CREATED,
    responses={
        400: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        404: {"model": ErrorResponse},
    },
)
def create_pattern(
    data: PatternCreateRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    """새 빈 패턴 만들기 (FR-2196) — OWNER·EDITOR 만."""
    return pattern_service.create_pattern(db, user, data)
```

```
@router.put(
    "/{pattern_id}/notes",
    response_model=PatternResponse,
    responses={403: {"model": ErrorResponse}, 404: {"model": ErrorResponse}},
)
def replace_notes(
    pattern_id: int,
    data: NotesSaveRequest,
    user: CurrentUser,
    db: Db,
) -> PatternResponse:
    """노트 통째 교체 (FR-231) — OWNER·EDITOR 만 (VIEWER 는 403)."""
    return pattern_service.replace_notes(db, user, pattern_id, data)


@router.delete(
    "/{pattern_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={
        400: {"model": ErrorResponse},
        403: {"model": ErrorResponse},
        404: {"model": ErrorResponse},
    },
)
def delete_pattern(
    pattern_id: Annotated[int, Path(ge=1)],
    user: CurrentUser,
    db: Db,
):
    """패턴 삭제 (FR-2196) — OWNER·EDITOR 만. 클립이 쓰고 있으면 400."""
    pattern_service.delete_pattern(db, user, pattern_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

목록과 상세는 다음과 같이 서비스에 넘기고 응답 모델을 그대로 돌려줍니다.

```
@router.get("", response_model=PatternListResponse)
def list_patterns(
    user: CurrentUser,
    db: Db,
    project_id: Annotated[int, Query(ge=1)],
) -> PatternListResponse:
    """프로젝트의 패턴 목록 (FR-233) — 멤버면 누구나. 노트는 담지 않는다."""
    return pattern_service.list_patterns(db, user, project_id)


@router.get(
    "/{pattern_id}",
    response_model=PatternResponse,
    responses={404: {"model": ErrorResponse}},
)
def get_pattern(pattern_id: int, user: CurrentUser, db: Db) -> PatternResponse:
    """패턴 한 건과 노트 전부 (FR-232) — 피아노롤이 열 때 쓴다."""
    return pattern_service.get_pattern(db, user, pattern_id)
```

문법 풀이

식	뜻
`@router.post("", ...)`	접두사 `/patterns` 와 붙어 `POST /patterns` 가 됩니다. 빈 경로도 맞습니다
`Annotated[int, Query(ge=1)]`	쿼리 `project_id` 는 1 이상만 받습니다
`status_code=status.HTTP_201_CREATED`	만들기는 본문과 함께 201 로 보냅니다
`NotesSaveRequest`	`{ "notes": [ … ] }` 본문을 검증합니다. 노트 칸 범위가 어긋나면 422입니다
`responses={400: ..., 403: ..., 404: ...}`	문서에 잘못된 요청과 권한, 없음의 모양을 적어 둡니다
`status.HTTP_204_NO_CONTENT`	삭제는 내용 없는 204 로 보냅니다
`pattern_id: Annotated[int, Path(ge=1)]`	경로의 패턴 번호는 1 이상만 받습니다
`CurrentUser`, `Db`	로그인 사용자와 DB 세션을 자동으로 받는 별칭입니다

3. 왜 이렇게

선택	이유
여섯 길을 한 파일에 둠	패턴 하나가 주는 자원이라 `patterns` 아래에 모아 두면 문서와 화면에서 찾기 쉽습니다
노트 저장을 PUT 으로 둠	피아노롤은 지금 보이는 노트 전부를 다시 보내므로, 부분 고치기가 아니라 통째 교체가 맞습니다
만들기와 노트 저장에 403 응답을 표시	VIEWER 는 쓰기를 못 하므로 403이 실제로 날 수 있음을 문서에 적어 둡니다
삭제에 400 응답을 표시	클립이 쓰는 패턴은 지울 수 없으므로 400이 실제로 날 수 있음을 문서에 적어 둡니다
목록과 상세의 응답 모델을 따로 둠	목록은 노트 없이 가볍고, 상세는 노트를 담습니다 (279 문서)
라우터를 얇게 둠	권한, 404, 클립 사용 검사는 서비스 한 곳에 있으므로 라우터는 부르기만 합니다

4. 검증

`tests/test_pattern_routes.py` 가 이 파일을 직접 부릅니다. `test_pattern_routes_registered` 가 `app.openapi()["paths"]` 에 여섯 경로가 실렸는지 확인하고, `test_create_pattern_201` 은 만들기 201, `test_create_pattern_viewer_is_403` 는 VIEWER 403, `test_replace_notes` 와 `test_replace_notes_validates_pitch` 는 노트 교체와 범위 검사를, `test_delete_pattern_204` 와 `test_delete_pattern_used_by_clip_is_400` 은 삭제 204 와 클립 사용 400 을 확인합니다. `test_list_patterns`, `test_get_pattern_with_notes`, `test_patch_pattern`, `test_non_member_gets_404`, `test_viewer_cannot_save_notes` 도 모두 통과합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
화면 확인	패턴 만들기, 피아노롤 노트 저장, 패턴 삭제를 시퀀서에서 직접 확인합니다